Privacy-Preserving and Trustworthy Cyber-Infrastructures

CIS 4212/6214 (Spring 2026)

Instructor: **Dr. Attila A. Yavuz**
TA: ***Saif E. Nouma***

Mini-Project: One-Time Signatures (OTSs)

(Due: April. $21^{nd}, 2026 - 23:59$) [100 Points]

(Upload your solutions to the CANVAS)
Requirements: HW groups are 2-3 people. Each student should contribute all parts of the answers. If one group member drops, the others are equally responsible (each HW can be handled by a single student if needed, so no excuse for missing a partner (if you are doing in group of 2), see syllabus for breaking-up with pair programming partner). Directly borrowing (e.g., copy-paste) from any material and putting in solutions (e.g, from online solutions, Wikipedia, or research papers) is plagiarism (see Syllabus for its corresponding actions). Please cite very carefully each resource you use but citing a solution does not give a license to directly put it as an answer. **All of your answers must be in your own words and interpretations. All your solutions will be passed through MOSS and Turnitin for plagiarism check.**
Note: Many of these questions require you to recall explanations/discussions during the class (this is why attendance is important), and also make research on the internet to find good answers. Please kindly note that very brief and/or informal explanations will potentially be not sufficient.
Remark: HW should be prepared by a text editor (e.g., Microsoft Word or Latex). Handwritten submissions are not accepted. In your answer key, please indicate the point of each question as it is done in this HW document. Preferably, take questions from here, plug them into your answer key with their sub scores, and provide your answer below them. This will facilitate the grading.
As indicated in Syllabus, **no partial credit is possible for coding answers that does not produce correct results with respect to test vectors, deviate from the description and rubric, are coded on platforms other than specified, or do not work on the grader's machine.**

Common Notes:

- We have provided a script and test files that you could use to test the correctness of your final codes (alice and bob). Please, only submit your code if it is functional.
- During grading, the seed and message will be randomly chosen, and your code must handle this variability correctly. We will also test your code with incorrect or partially correct signature elements, so it's essential that your code is robust in such scenarios. To facilitate testing under these conditions, please ensure that your code can handle incorrect public keys or signature as input.
- The code that throws errors or does not successfully compile/run on our side, receives zero credit (It doesn't matter if it works on your computer!).
- You can check your answer with each provided test file manually or you can use the script to verify your solution for the whole project. When checking the files manually, make sure you are not changing the files, because even a single null pointer at the end would show invalid when testing with the script.

Programming Assignment 1 : Lamport OTS [50 points]

For detailed description of Lamport OTS, refer to the class slides and documents

Implementation Setting

- We are implementing Lamport OTS Variant I.
- For the required one-way function f, use SHA-256 from OpenSSL (*i.e.*, $n = 256$).
- For the cryptographic hash function $Hash$, also use SHA-256 from OpenSSL.
- According to variant I (Slide 5,9), both the SK and PK sizes would be $2n^2$ bits, while signature is of size n^2.
- For the correctness of your code, please, use unsigned char for all operations. For instance, since the content of the **"SK.txt"** and **"PK.txt"** files are Hex strings, you must convert them to unsigned char first, and then use it in the signing and verifying processes.

Lamport Key Generation: Certificate Authority [20 pt]

Input Arguments

- The Seed filename **"Seed$i.txt"**

Certificate Authority (CA) Performs the following steps to generate secrete and public key sets:

1. CA reads the seed from a file named **"Seed.txt"**.
2. According to Lamport signature, SK is a matrix with dimension $2 \times n$ where each element is also n bits (256 bits). In order to generate SK, CA appends two bytes at the end of the seed, where the row index is $\{0, 1\}$ and the column index is $\{0, 1, ..., 255\}$. Specifically, the SK elements are generated by using ChaCha20 PRNG on the seed, concatenated with i and j as follows:

$$\text{SK}[i, j] \rightarrow \text{PRNG}(\text{seed}\|i\|j) \quad \text{for } i \in \{0, 1\} \text{ and } j \in \{0, 1, \ldots, 255\}$$

3. Subsequently, CA generates the PK elements using SHA-256.
4. CA writes the SK elements in a file named **"SK.txt"**. Write in the SK.txt file line by line in a column-wise order, i.e., start from element $(0, 0)$, then $(1, 0)$, then go to the second column, $(0, 1)$ and $(1, 1)$, then, $(0, 2)$ and $(1, 2)$, and so on to $(0, 255)$ and $(1, 255)$. (See the test vectors to understand better.)
5. Similar to the last step, CA writes the PK elements in a file named **"PK.txt"**. (Write in the PK.txt file line by line in a column-wise order, similar to SK)

Compile the key generation part with the following command:
gcc KeyGen.c -lcrypto -o CA
Run your code for the first test files with the following commands:
./CA Seed$i.txt

Lamport Signing: Alice [15 pt]

Input Arguments

- Message filename

Alice plays the role of a signer as follows:

1. Alice reads the secret key from **"SK.txt"**. Note that SK.txt file *must* contain the hex value of the secret key.
2. Alice reads the message from the provided filename. Read the message as unsigned char.
3. Alice compute the hash on the message using SHA256.
4. According to Lamport OTS variant I (see Slide 6), Alice signs the hash value bit by bit.
5. Alice writes the signature in Hex format in **"Signature.txt"**. (Write line by line as shown in the test vectors.)

Terminal
Compile the signers part with the following command:
gcc Sign.c -lcrypto -o Alice
Run your code for the first test files with the following commands:
./Alice Message.txt

Lamport Verification: Bob [15 pt]

Input Arguments

- Message filename

Bob plays the role of a verifier as follows:

1. Bob reads the public keys from **"PK.txt"**. Note The PK.txt file contains Hex value of the public key.
2. Alice reads the message from the provided filename. Read the message as unsigned char.
3. Bob hashes the message using SHA256.
4. According to Lamport OTS variant I (see Slide 6), Bob verifies the signature on the hash value, bit by bit.
5. If the above holds for all bits, Bob writes "Signature is Valid" in a file named "Verification.txt"; otherwise, he writes "Verification is Invalid" in the file.

Terminal
Compile the verifier part with the following command:
gcc Verify.c -lcrypto-o Bob
Run your code for the first test files with the following commands:
./Bob Message.txt

Programminig Assignment - Notes:

- There are three parties: The singer (Alice), the verifier (Bob), and the key generator (Certificate Authority). The CA generates the SK and PK sets and writes in files and there are two terminals that we will refer to as "Alice" and "Bob" as the sender and receiver, respectively.

- **You Must submit 3 files to canvas:**
  KeyGen.c , Sign.c , Verify.c

Rubric:

- Correct **SK.txt** file **[10 pt]**
- Correct **PK.txt** file **[10 pt]**
- Corret **Signature.txt** file **[15 pt]**
- Correct **Verification.txt** file + Successful Verification + Rejection of Invalid Signature **[15 pt]**

Programming Assignment 2: HORS OTS [50 points]

For detailed description of HORS Signature, refer to the class slides and documents

Implementation Setting

- We are implementing HORS OTS, $r = 1$ (i.e., sign *only one message* under a given private/public key pair).
- For the required one-way function f, use SHA-256 from OpenSSL.
- For the cryptographic hash function $Hash$, also use SHA-256 from OpenSSL.
- The parameter t for the HORS scheme, which determines the size of the secret key (SK) and public key (PK), will be selected as a power of 2, ranging from 128 to 1024.
- The parameter k, which determines the signature size, is chosen based on t s.t. $k \cdot \log_2(t) \leq 256$. Specifically, choosing $t = 128$ and $t = 1024$ will result in $k = 64$ and $k = 25$, respectively. Although this approach *may* impact security due to discarding a few bits of the digest, it ensures consistency throughout your implementation.
- All SK elements are 256-bit data. Consider SHA-256 as a one-way function, all PK elements are also 256-bit.
- For the correctness of your code, please, use unsigned char for all operations. For instance, since the content of the **"SK.txt"** and **"PK.txt"** files are Hex strings, you must convert them to unsigned char first, and then use it in the signing and verifying processes.

HORS Key Generation: Certificate Authority [20 pt]

Input Arguments

- 30 Bytes of Seed in a file named **"Seed.txt"**.
- The parameter t as an integer.

Certificate Authority (CA) Performs the following steps to generate secrete and public key sets:

1. CA reads the seed from a file named **"Seed.txt"** containing 30 characters (30 Bytes).
2. According to HORS signature, CA generates t random $l-$bit strings. In our setting, $l = 256$ bits.

3. The secret key SK is generated by applying the ChaCha20 PRNG to a structured seed input. Formally, the SK elements are derived as:
$$\text{SK} \leftarrow \text{PRNG}(\text{seed}\|i) \quad \forall i \in \{0, \ldots, t-1\}$$

4. Subsequently, CA generates the PK elements using SHA-256 evaluations on the SK elements.

5. CA writes the SK elements in Hex format in a file named **"SK.txt"**. Please, write in the SK.txt file line by line as shown in the test vectors.

6. Similar to the last step, CA writes the PK elements in Hex format in a file named **"PK.txt"**. Write in the PK.txt file line by line, similar to SK.

Terminal
Compile the key generation part with the following command:
gcc KeyGen.c -lcrypto -o CA
Run your code for the first test files with the following commands:
./CA Seed.txt t

HORS Signing: Alice [20 pt]

Input Arguments

- Message filename.
- The HORS parameter t as an integer.
- The HORS parameter k as an integer.

Alice plays the role of a signer as follows:

1. Alice reads the message form a file named, **"Message.txt"**.

2. Alice reads the secret keys from **"SK.txt"** file generated by the CA. Note that SK.txt file must contain Hex value of the secret key and needs to be converted to unsigned char prior to any operation.

3. Alice hashes the message using SHA256.

4. According to HORS signing process, Alice splits the hash value into k binary sub-strings of size $log_2(t)$. Subsequently, interpret each $log_2(t)$ binray array as an integer, and use that integer to create the signature.

5. Alice writes the signature in Hex format in a file named **"Signature.txt"**. Write in a file line by line.

Terminal
Compile the signers part with the following command:
gcc Sign.c -lcrypto -o Alice
Run your code for the first test files with the following commands:
./Alice Message.txt t k

HORS Verification: Bob [10 pt]

Input Arguments

- Message filename.
- The HORS parameter t as an integer.
- The HORS parameter k as an integer.

Bob plays the role of a verifier as follows:

1. Bob reads the public keys from **"PK.txt"** file generated by the CA. (The PK.txt file contains Hex value of the public key and needs to be converted to unsigned char for any operation.)
2. Similar to the signing process, Bob hashes the message using SHA256.
3. Exactly like the signer part, Bob splits the hash value into k binary sub-strings of size $log_2(t)$. Subsequently, interpret each $log_2(t)$ as an integer, and use that integer to verify the signature.
4. If the above holds for all k signature elements, Bob writes "Signature is Valid in a file named "Verification.txt"; otherwise, he writes "Signature is Invalid". Check the test vectors to see the format of the verification file.

Terminal
Compile the verifier part with the following command:
gcc Verify.c -lcrypto -o Bob
Run your code for the first test files with the following commands:
./Bob Message.txt t k

Programminig Assignment - Notes:

- There are three parties: The singer (Alice), the verifier (Bob), and the key generator (Certificate Authority). The CA generates the SK and PK sets and writes in files and there are two terminals that we will refer to as "Alice" and "Bob" as the sender and receiver, respectively.
- In order to use the script, just put your codes, the rest of the provided files along with the "VerifyingYourHORSSolution.sh" in one folder and run the following command in the terminal:
  bash VerifyingYourHORSSolution.sh
- **You Must submit 3 files to canvas:**
  KeyGen.c , Sign.c , Verify.c

Rubric:

- Correct **SK.txt** file **[15 pt]**
- Correct **PK.txt** file **[5 pt]**
- Correct **Signature.txt** file **[20 pt]**
- Correct **Verification.txt** file + Successful Verification + Rejection of Invalid Signature **[10 pt]**

Extra-Credit Question [15 points]

0.1 Winternitz OTS (W-OTS) [5 points]

Formally describe the W-OTS digital signature scheme [1, 2] and how it evolve from Lamport OTS. Your answer must define the Winternitz parameter w, the key generation procedure, the signing algorithm over message digest $m \in \{0,1\}^n$, and the verification procedure. Using your formal description, analyze the trade-off between signature size and the number of hash evaluations as a function of w .

0.2 Forest of Random Subsets (FORS) [5 points]

Formally describe the FORS digital signature scheme [6], introduced as a component of SPHINCS$^+$ [6]. Your answer must specify the parameter tuple (n, k, t), define the secret and public key structures, and present the signature algorithms (key generation, signature generation, and signature verification), and $\mathsf{FORS.Verify}$. Contrast FORS with HORS and justify why FORS is classified as a *few-time* rather than *one-time* signature scheme.

0.3 Hash-based Standards: stateful XMSS and Stateless SLH-DSA [5 points]

a. Formally describe how XMSS [7] incorporates W-OTS$^+$ [3] to construct a stateful many-time signature scheme.
b. Formally describe how SPHINCS$^+$ [9, 6] combines FORS [6], W-OTS$^+$ [3], and XMSS to construct a stateless signature scheme with practically unlimited number of signing capabilities.

References

[1] R. C. Merkle, "A certified digital signature," in *Proc. CRYPTO 1989*, LNCS vol. 435, pp. 218–238, Springer, 1989. [W-OTS first described here as a generalization of Lamport OTS.]

[2] A. Hülsing, "On the security of the Winternitz one-time signature scheme," in *Proc. AFRICACRYPT 2011*, LNCS vol. 6737, pp. 363–378, Springer, 2011. ePrint: `https://eprint.iacr.org/2011/191`

[3] A. Hülsing, "W-OTS+ – Shorter signatures for hash-based signature schemes," in *Proc. AFRICACRYPT 2013*, LNCS vol. 7918, pp. 173–188, Springer, 2013. ePrint: `https://eprint.iacr.org/2017/965`

[4] L. Reyzin and N. Reyzin, "Better than BiBa: Short one-time signatures with fast signing and verifying," in *Proc. ACISP 2002*, LNCS vol. 2384, pp. 144–153, Springer, 2002. ePrint: `https://eprint.iacr.org/2002/014`

[5] D. J. Bernstein, D. Hopwood, A. Hülsing, T. Lange, R. Niederhagen, L. Papachristodoulou, M. Schneider, P. Schwabe, and Z. Wilcox-O'Hearn, "SPHINCS: Practical stateless hash-based signatures," in *Proc. EUROCRYPT 2015*, LNCS vol. 9056, pp. 368–397, Springer, 2015.

[6] D. J. Bernstein, A. Hülsing, S. Kölbl, R. Niederhagen, J. Rijneveld, and P. Schwabe, "The SPHINCS+ signature framework," in *Proc. ACM CCS 2019*, pp. 2129–2146, ACM, 2019. ePrint: `https://eprint.iacr.org/2019/1086` [Introduces FORS and tweakable hash functions.]

[7] A. Hülsing, D. Butin, S. Gazdag, J. Rijneveld, and A. Mohaisen, "XMSS: eXtended Merkle Signature Scheme," IETF RFC 8391, May 2018. `https://www.rfc-editor.org/rfc/rfc8391`

[8] D. Cooper, D. Apon, Q. Dang, M. Davidson, M. Dworkin, and C. Miller, "Recommendation for stateful hash-based signature schemes," NIST Special Publication 800-208, October 2020. `https://csrc.nist.gov/pubs/sp/800/208/final`

[9] National Institute of Standards and Technology, "Stateless hash-based digital signature standard (SLH-DSA)," Federal Information Processing Standard 205, August 2024. `https://csrc.nist.gov/pubs/fips/205/final`